

Discussion 3: Opening the Black Box

Interpretability, Ablation, and Why It Matters

Gamut Technology Services · AI Engineering Academy · Week 4, Module 01

Format: facilitated group discussion · Target duration: 60 minutes

Companion slides: Module 01, slides 16 to 25

Discussions 1 and 2 built a picture of what the model does inside. This one is about how we check those claims and why it matters in production. The big idea is simple and familiar: to understand a system you did not fully design, you run controlled experiments on it. That is exactly what interpretability does to a language model.

What you should be able to do by the end

1. Explain logit analysis as reading a debugging signal from inside the model.
2. Define ablation as a controlled experiment: remove or change a component, measure the effect.
3. Compare zero, mean, and resample ablation, and activation patching, and say which question each one fits.
4. Explain the redundancy caveat: no measured effect does not prove a component is useless.
5. Connect interpretability to safety, debugging, and trust for a company deploying an LLM.

The scenario

SETUP: A CONFIDENT WRONG ANSWER

Timberline's assistant confidently told a customer the return window is 30 days. In this fictional world the real policy is 90 days. Leadership asks a sharper question than usual: not “patch the prompt,” but “can we find out why the model produced that, so we can trust the fix?”

That is a debugging question and a trust question at the same time. Interpretability is the toolkit. Today we reason about how we would investigate it.

Segment A. Logits as a debugging signal

Slide 16. The final answer alone does not tell you why the model chose it. The logits do. They are the raw scores the model assigned to each candidate token before it picked one. Reading them is like reading logs or instrumentation: you look behind the output to see what the system was actually weighing.

The deck's worked example: ask “Capital of France?” and watch the logit for “Paris.” If removing a particular attention head makes that logit drop sharply, that head clearly mattered for the recall. The logit is the measurable signal that lets you make a causal claim.

DISCUSSION PROMPT A

The assistant answered “30 days.” Imagine you can inspect the model's logits at the answer position.

- How would you tell whether the model nearly said 90 (a close race between 30 and 90) versus was confidently wrong (30 far ahead of 90)?
- Why does that difference change how you would fix it?
- What does the confident-wrong case suggest about where the problem lives?

QUICK CHECK A

In one sentence: what do logits let you see that the final answer alone does not?

Segment B. Ablation is a controlled experiment

Slide 17. Ablation means systematically removing or changing a component to learn what it does. You knock out a head, a layer, or a neuron, then watch how behavior changes. If performance drops a lot, the component mattered. If nothing changes, it may not, subject to a caveat we will get to.

Engineers do versions of this constantly: comment out a line and see what breaks, toggle a feature flag off, disable a service and watch the dashboard, or run git bisect to localize a regression. Ablation is that same discipline, one change at a time, measure the delta, applied to model internals.

DISCUSSION PROMPT B

You suspect one specific component is responsible for the model copying the policy number out of the provided context. Design a clean experiment using only the move “remove the component and measure the change.”

- What is your control condition?
- What is your intervention?
- What exactly do you measure to decide whether that component was responsible?

Natural break point. Take five minutes before Segment C.

Segment C. Four ways to ablate, and their trade-offs

Slide 18. “Remove” is not one thing. How you knock a component out changes what you learn and what artifacts you introduce.

- **Zero ablation.** Set the component's output to zero. Simple, but the model never saw exactly-zero during training, so this can create odd off-distribution effects unrelated to what you are studying.
- **Mean ablation.** Replace the output with its average value across a dataset. More realistic than zero, fewer artifacts, because a typical value is something the model has effectively seen.
- **Resample ablation.** Replace the output with the value it took on a different input. This tests whether the component encodes something specific to this input or something general.

- **Activation patching.** Run a clean input and a corrupted input, then patch clean values into the corrupted run, position by position, to localize where the important information is processed. This is the workhorse for pinning down where something happens.

A rough engineering parallel: zero ablation is yanking a dependency entirely, mean ablation is swapping in a default or typical response, resample ablation is feeding in another request's value, and activation patching is bisecting by dropping known-good state into a failing run until you find the spot that flips the result.

DISCUSSION PROMPT C

Match each question to the ablation you would reach for, and justify it:

- “Does this head matter at all for the task?”
- “Is this head encoding something specific to THIS input, or something general?”
- “Where in the network does the policy number actually get processed?”

Where more than one method could work, say what each choice would cost you.

Segment D. What ablation reveals, and why a retailer cares

Slide 19. Ablation reveals component importance, redundancy, and information flow. The redundancy point is the trap worth internalizing. Models often learn the same job in more than one place. Knock out one head and behavior may not budge, not because that head is useless, but because a twin covered for it. Ablate both together and the effect appears.

The parallel is load-balanced servers. Take one of two redundant servers offline and the site stays up. That does not mean the server did nothing. It means there was redundancy. Same logic, same caution.

Slides 20 and 22 close the loop. Interpretability connects high-level behavior down to low-level computation, and it matters in production for concrete reasons: catching failure modes before deployment, debugging root cause instead of guessing, and building trust that the system does what you intend. This is the responsible-AI spine of the whole module. When Timberline ships an assistant that talks to customers, being able to ask why it said something is not academic. It is how you keep it safe and accountable.

DISCUSSION PROMPT D

You ablate head A and see no change. A teammate concludes: “Head A does nothing. Safe to ignore.”

- What is the flaw in that conclusion?
- How would you test whether head A really is doing nothing?
- Now zoom out: give one concrete reason a retailer deploying this assistant should care about any of this interpretability work at all.

QUICK CHECK D

True or false: if ablating a component causes no performance drop, the component is definitely unimportant.

Synthesis across all three discussions

Slide 20 shows the interpretability stack: behaviors on top, then logits and predictions, then layer outputs, then attention patterns, then the Query, Key, and Value computations at the base. Look at what you climbed. Discussion 1 built the base, attention and QKV. Discussion 2 covered the middle, specialized heads and the path to logits and predictions. Discussion 3 connected the top, using logits and ablation to explain behavior. You now have a working map of the whole stack, from the fundamentals up.

QUESTIONS TO CARRY INTO THE DEEP-DIVE SESSION

- Which specific heads matter most for the task I care about?
- How do heads combine to produce more complex behavior?
- Which ablation best isolates the effect I want to measure?
- How would I visualize attention patterns to build intuition?
- What are the limits and failure modes of these interpretability techniques?

Quick check answers

1. **A.** Logits show the raw scores the model assigned to each candidate token before choosing, so you can see what it was weighing and how close the race was, which the single final answer hides.
2. **D.** False. Redundancy can hide a component's role. No measured effect from removing one component does not prove it is unimportant. Test by ablating it together with likely twins.